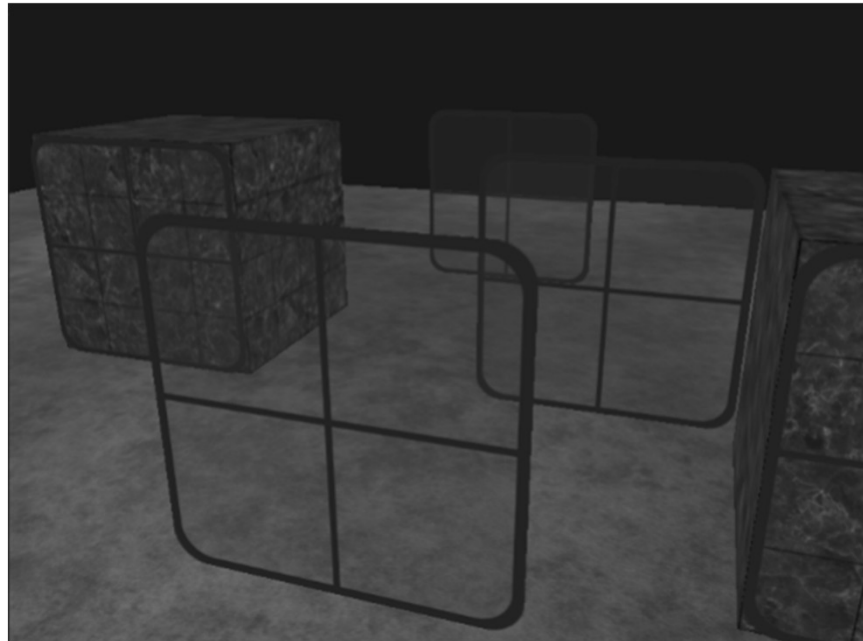


Lecture 5 — Blending (Depth Buffer & Color Blending)

- Module focus: composing multiple contributions at a pixel (translucency, particles, UI)
- Key systems: depth buffer (visibility) + blending (color/coverage)

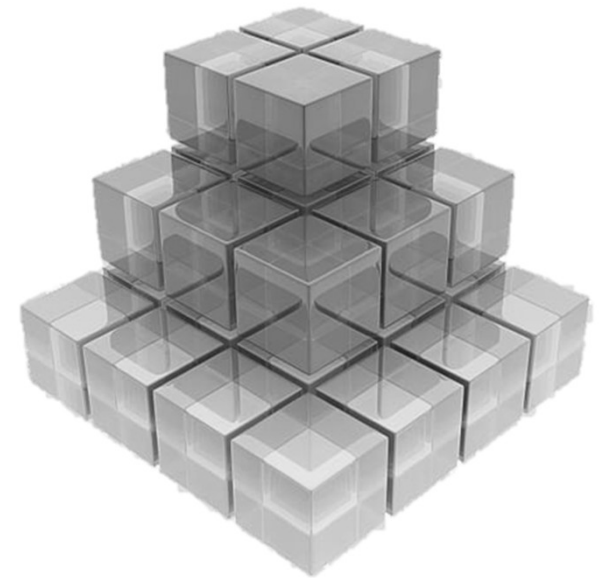
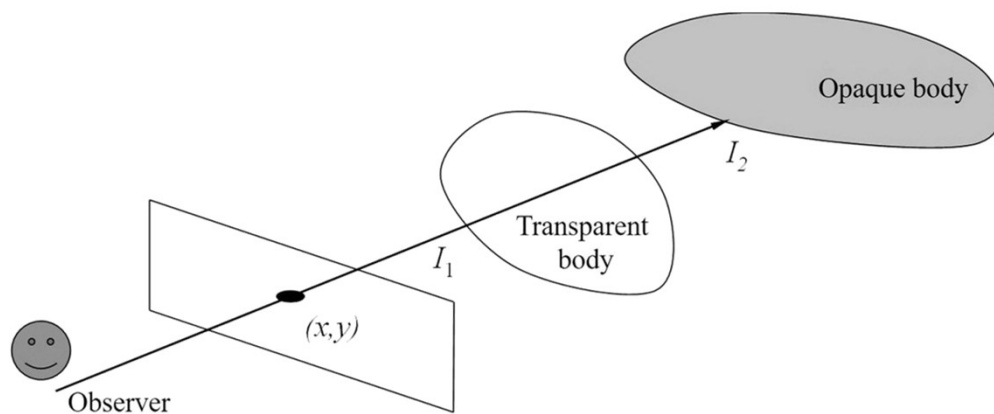


Learning Outcomes & Session Plan

- Explain blending concepts: depth buffer vs. color blending (and why both matter)
- Improve design ability: choose appropriate blend modes for a desired visual effect
- Process graphics via programming tools: implement correct real-time blending in code
- Plan: Theory (65 min) → Break (5 min) → Practice lab (65 min) → Wrap-up (5 min)

Why Blending? (What “Translucency” Means on Screen)

- Translucency: multiple scene locations contribute to one screen-space sample
- Because light contributions are independent, we can sum them (superposition)
- Real-time approximation: draw surfaces separately and “compose” them by blending
- Common cases: glass, smoke/particles, decals, UI overlays



Raster Pipeline Context: Depth Test vs. Blending

- Rasterization generates fragments → per-fragment tests run (e.g., depth test)
- If the fragment passes visibility tests, it can update the framebuffer
- Blending combines the incoming (source) color with existing (destination) color
- Order matters when blending is enabled: “what was already there” affects the result

Depth Buffer Refresher (Z-Buffer = Visibility)

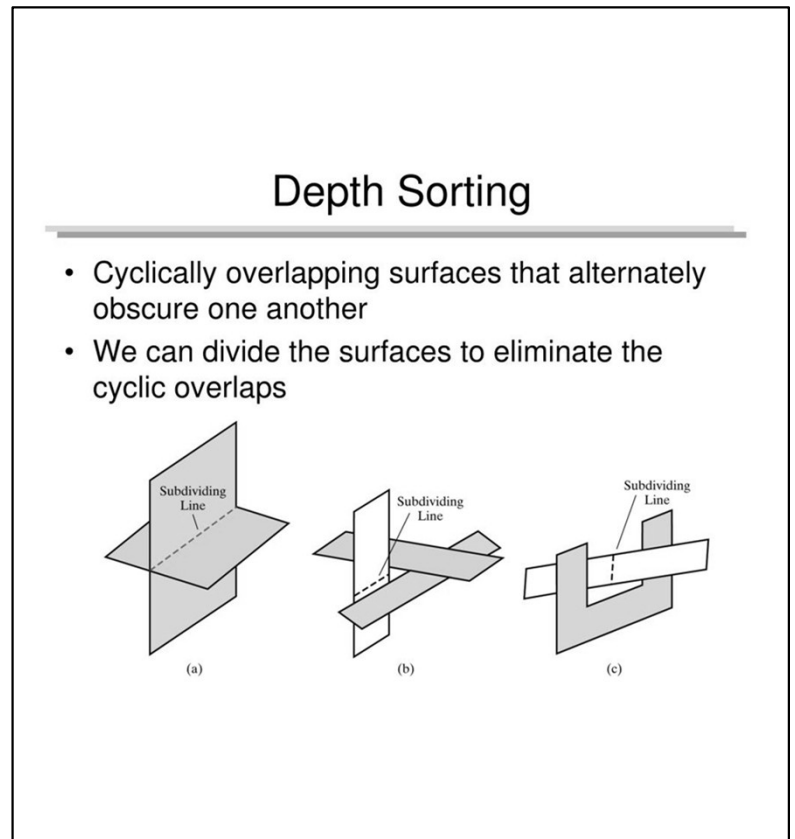
- Depth buffer stores the closest depth value seen so far per pixel
- Depth test compares the new fragment's depth vs. stored depth (e.g., LESS)
- Clear depth each frame (and choose near/far wisely for precision)
- Depth test and blending can both be enabled—this is common for transparency



Example: visualized depth buffer (grayscale)

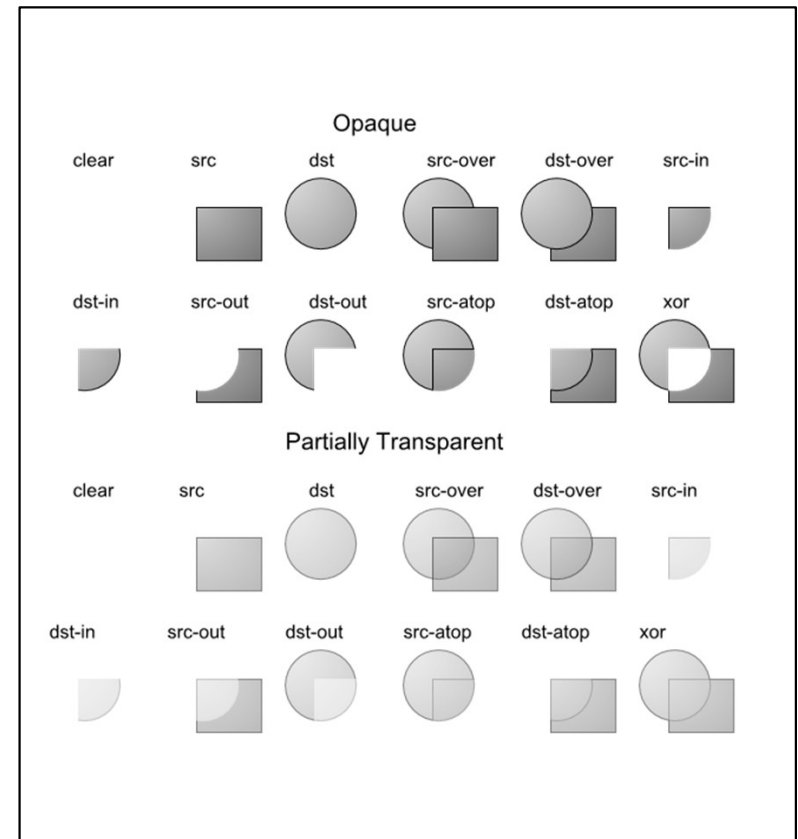
Why Transparency Is Hard: Depth + Order Dependence

- With blending, the “correct” result depends on drawing back-to-front (Painter’s Algorithm)
- If transparent objects write depth, they can incorrectly block farther transparent objects
- Intersecting/overlapping transparent geometry can create cycles (no perfect sort)
- We mitigate with: sorting, disabling depth writes, or approximate OIT methods



Blending as a Function: Source + Destination → New Destination

- Per color channel: $d = \sigma(s,d) \cdot s + \delta(s,d) \cdot d$
- BlendFunc selects σ and δ from a limited set (for GPU efficiency)
- Common factors: ZERO, ONE, SRC_COLOR, DST_COLOR, SRC_ALPHA, (1-SRC_ALPHA)
- Think “weights” for how much of source and destination are kept



Porter–Duff compositing examples (source/destination)

OpenGL Controls: Enabling and Configuring Blending

- Enable/disable: `glEnable(GL_BLEND)` / `glDisable(GL_BLEND)`
- Choose factors: `glBlendFunc(srcFactor, dstFactor)`
- Choose equation: `glBlendEquation(GL_FUNC_ADD / SUBTRACT / ...)`
- Advanced: separate RGB/alpha factors (`glBlendFuncSeparate`) + multiple render targets

The “Over” Operator (Classic Alpha Blending)

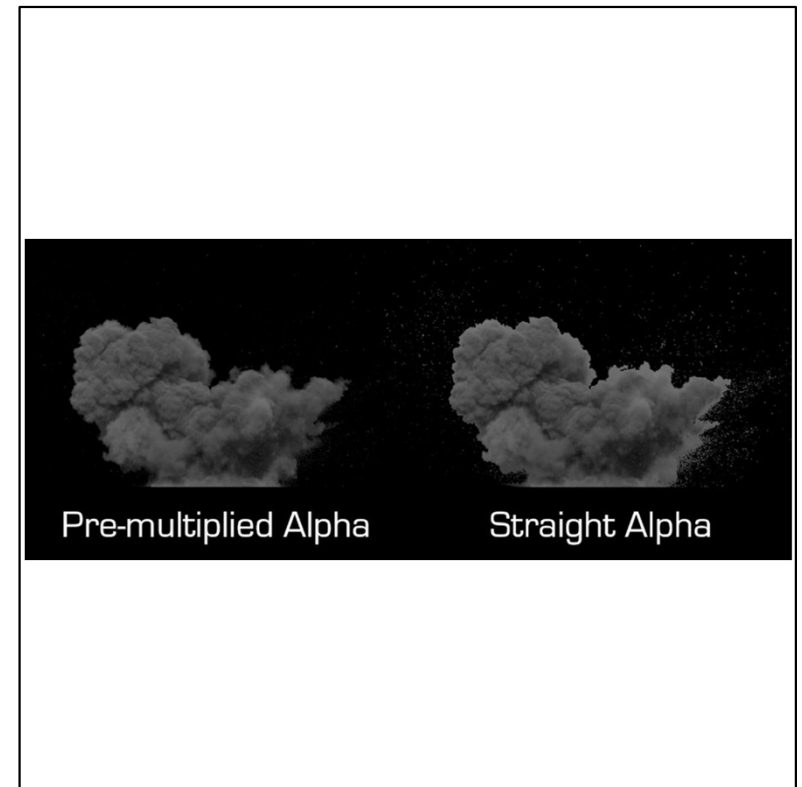
- Standard compositing (source over destination): $d = \alpha \cdot s + (1 - \alpha) \cdot d$
- Interpret α as *coverage fraction* at the pixel (0..1), not “magic transparency”
- Requires back-to-front rendering (near objects composited over far objects)
- Default for sprites, UI, decals, and many particle systems

Alpha as Partial Coverage (Anti-Aliasing Intuition)

- Binary mask (0/1) causes jagged edges with 1 sample per pixel (aliasing)
- Fractional α estimates “how much of this pixel is covered” by the shape
- Filtering/MIP-maps can precompute partial coverage for textures
- Beware correlation: some alpha-to-coverage patterns can look wrong when layers overlap

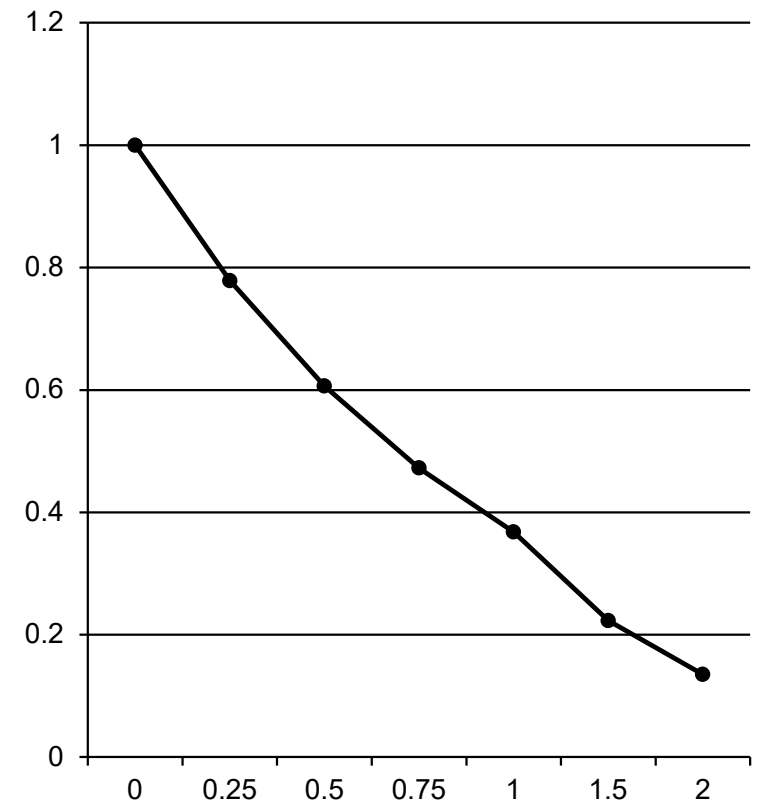
Premultiplied Alpha (Avoiding Halos and Getting Correct Math)

- Store colors as $(\alpha R, \alpha G, \alpha B, \alpha)$ instead of (R, G, B, α)
- Benefits: fewer multiplies during compositing + avoids edge fringes (“halos”)
- Blend factors typically: $(ONE, ONE_MINUS_SRC_ALPHA)$ for premultiplied textures
- Rule of thumb: match your asset format to your blend mode (don’t premultiply twice)



Transmission & Thickness: Tint + Exponential Attenuation

- Tinted glass needs per-channel transmission t_{RGB} (it filters background light)
- Thicker material transmits less: transmission decreases exponentially with distance
- Simple model: $t = e^{(-k \cdot x)}$, where x is thickness along the ray
- In practice: approximate x (constant, or front/back depth difference)



Example attenuation curve ($k = 1$)

Common Blend Modes (When to Use Which)

- Alpha “over”: (SRC_ALPHA, ONE_MINUS_SRC_ALPHA) → natural layering/coverage
- Additive: (ONE, ONE) → glow, sparks, bloom, lens flare (adds light)
- Multiply-like: (ZERO, SRC_COLOR) on destination → darkening / transmission tricks
- Pick blend mode to match intent: coverage vs. emission vs. filtering

Pitfalls & Debugging Clues (Theory)

- Overbright overlaps: additive blending or repeated “over” without intent
- Wrong hue: confusing coverage α with transmission (t) or mixing color spaces
- Missing/miscolored shadows: translucency model doesn't match light transport
- Sorting failures: intersecting transparent geometry creates cyclic occlusion

Concept Check (5–7 minutes)

- Q1: Depth buffer vs. blending — what problem does each solve?
- Q2: What BlendFunc gives “overwrite” behavior? (Hint: ONE, ZERO)
- Q3: Why must classic alpha blending be drawn back-to-front?
- Q4: When do you prefer premultiplied alpha over straight alpha?

Practice Lab Overview (Guided, Step-by-Step)

- Goal: implement correct opaque + transparent rendering passes
- You need: a starter OpenGL/WebGL project + a few textures (with alpha)
- Workflow: implement → run → observe artifact → fix by changing state/order
- Deliverable: screenshot + short explanation of your chosen blend modes

Practice Step 0 — Enable Depth Testing

- Request a depth buffer when creating the framebuffer/window
- Enable test: `glEnable(GL_DEPTH_TEST)` and choose `glDepthFunc(GL_LESS)`
- Clear each frame: `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)`
- Verify with two overlapping cubes: nearer cube should always win

Practice Step 1 — Opaque Pass (Depth Write ON, Blend OFF)

- Disable blending for opaque geometry (default: GL_BLEND off)
- Depth test ON + depth write ON (glDepthMask(GL_TRUE))
- Render opaque objects in any order (depth test resolves visibility)
- Only after this pass: start rendering transparent objects

Practice Step 2 — Classic Alpha Blending (Straight Alpha)

- Enable blending: `glEnable(GL_BLEND)`
- Blend equation: `glBlendEquation(GL_FUNC_ADD)`
- Blend factors: `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`
- Test with a semi-transparent quad ($\alpha = 0.5$) over a colored background

Practice Step 3 — Observe Order Dependence

- Render 3 transparent quads with different colors and $\alpha \approx 0.5$
- Change draw order (front-to-back vs back-to-front) and compare results
- Explain the difference using: result depends on the current destination color
- Conclusion: blending is not commutative $\rightarrow$ sort transparent geometry

Practice Step 4 — Sort Transparent Objects (Back-to-Front)

- Compute a depth key per object: distance to camera (or view-space z)
- Sort descending (far $\rightarrow$ near) before rendering transparent objects
- Render transparent after opaque pass (still depth-tested)
- Note: this is object-level sorting (per-triangle sorting is expensive)

Practice Step 5 — Keep Depth Test, Disable Depth Writes

- Keep depth testing enabled so transparent fragments behind opaque are rejected
- Disable depth writes during transparent pass: `glDepthMask(GL_FALSE)`
- Render sorted back-to-front with blending ON
- Restore state after: `glDepthMask(GL_TRUE)` (avoid “state leaks”)

Practice Step 6 — Premultiplied Alpha (Correct Edges)

- Premultiply RGB by α (offline in asset pipeline, or in shader)
- Switch blend factors: `glBlendFunc(GL_ONE, GL_ONE_MINUS_SRC_ALPHA)`
- Compare with straight alpha: look at soft edges and mipmapped textures
- Document: what artifact disappeared (e.g., dark/bright fringes)

Practice Step 7 — Additive Blending for Glow/Emission

- Use additive blend to “add light”: `glBlendFunc(GL_ONE, GL_ONE)`
- Render glow sprites/particles after opaque pass (often with depth writes off)
- Limit intensity to avoid washed-out images (clamp or tone-map later)
- Optional: blur bright parts and add back → simple bloom effect

Practice Step 8 — Tinted Glass via Destination Modulation

- Goal: background is filtered by transmission color t_{RGB} (like colored glass)
- Pass A: multiply destination by t using `BlendFunc(ZERO, SRC_COLOR)`
- Pass B: add surface reflection/highlights over it (e.g., `SRC_ALPHA, ONE`)
- This separates “transmission” (t) from “coverage” (α) for better control

Practice Step 9 — Thickness-Based Attenuation (exp in Shader)

- Approximate thickness x : constant, or $\text{backfaceDepth} - \text{frontfaceDepth}$
- Compute transmission: $t = \exp(-k \cdot x)$ per channel (k can be color)
- Use t to modulate background (previous step) for more realistic glass
- Experiment: increase k or $x \rightarrow$ darker/more saturated transmitted color

Practice Step 10 — Debug Checklist (Most Common Mistakes)

- State leaks: forgot to re-enable depth writes or disable blending for later draws
- Wrong asset assumption: straight alpha texture but premultiplied blend (or vice versa)
- Sorting: transparent pass rendered in the wrong order or mixed with opaque draws
- Framebuffer setup: no depth buffer attached, or depth not cleared each frame

Mini-Project (15–20 minutes): Design a Translucent Scene Element

- Choose ONE: (A) stained-glass window, (B) translucent UI panel, (C) smoke + glow particles
- Must use: opaque pass + transparent pass + at least one non-default blend mode
- Show 2 screenshots: before fix (artifact) and after fix (corrected)
- Explain your choices in 4–6 sentences (depth state + blend state + ordering)

Extension Topic: Order-Independent Transparency (OIT) in Practice

- Sorting fails for intersecting transparency → OIT aims to remove order dependence
- Exact OIT often stores per-pixel fragment lists (A-buffer family) — memory-heavy
- Popular approximation: Weighted Blended OIT (single pass, constant memory)
- Takeaway: know when sorting is “good enough” vs. when OIT is needed

Wrap-Up & Reading

- Recap: depth buffer = visibility; blending = composition (coverage/transmission/emission)
- Default real-time recipe: opaque pass → transparent pass (sorted, depth writes off)
- Reading: Hughes et al., Section 14.10 + Porter–Duff (1984) for compositing foundations
- Next lecture: advanced transparency/antialiasing techniques (and performance trade-offs)